

More Than Meets the Eye

Rishabh Singh

Can we use a transformer
to predict ratings based on
student reviews?

The Approach

At a glance

1. Ingest data from PlanetTerp
 2. Load transformer
 3. Tokenize input reviews
 4. Train and fine tune
 5. Use reviews to prompt model and predict ratings
-

The Data

—

Planet Terp

1. Hosts reviews for various UMD classes
2. API to ingest all reviews for any teacher
3. Chose 5 different teachers and used their reviews as our dataset
4. Total of 402 reviews



Extra Notes On Data

When picking professors I chose to go with all math teachers to add a little consistency to reviews as well as 3 teachers that whose averages were near the middle and one prof which great reviews and one with less than great reviews for a total of 402 reviews.

Professor	# of Reviews	Average Rating
Sana Jahedi	95	2.53
Manoussos Grillaki	83	2.83
Patrick Brosnan	49	2.59
Raluca Rosca	79	1.96
Archana Khurana	96	4.60

The Model

—

Phi 3

Microsoft SLM

- Quantized Based Model
 - Lowers GPU's RAM usage
 - BitsAndBytes Config
 - Allows flexibility and easier use of other models
 - LoRa Adapters
 - Allow model to still be trained
-

What Exactly?

Base Model

Most of the Phi-3 model is outside of my scope and simply used as a foundation, but I believe it uses a quantized model to cut down on parameters needed to be trained and bits and bytes config to keep the model flexible and easy to change. Low-Rank adapters needed to help train a quantized model were also set up alongside. This is the model that was used to train and then the model with updated weights after training was used for our final results.

Tokenizer

The tokenizer which was used to take our reviews and prompt into the transformer was pretty standard and came from the Auto Tokenizer library

Training

Supervised Fine-Tuning (SFT) Trainer

- Format Data
 - Follows Conversational format
- Hyperparameters
 - Epoch & Batch Size
- Prompt
 - 'Give a single number of stars for the following review between 1 and 5 but biased more towards the extremes of 1 and 5: '

Diving a Little Deeper

The training was done several times for different hyperparameters. Mainly epoch and batch size to determine how many passes the dataset did and how many reviews were used in one iteration respectively. Unsurprisingly, the process that minimized training loss most involved the highest epochs and batch sizes. The chart shows the loss with epoch of 10 and batch size of 12.

The training itself used the SFT trainer instead of pure Pytorch (sorry) to attempt at cutting down training time and simplify training format. It handled our data in the conversational format which took our “please rate” prompt as the system prompt and the student review as the user prompt. The training was then done and stored for future use

Step	Training Loss
170	1.05
180	0.838
190	0.844
200	0.733
210	0.640
220	0.612
230	0.508
240	0.446
250	0.471
260	0.374
270	0.339
280	0.350

Results

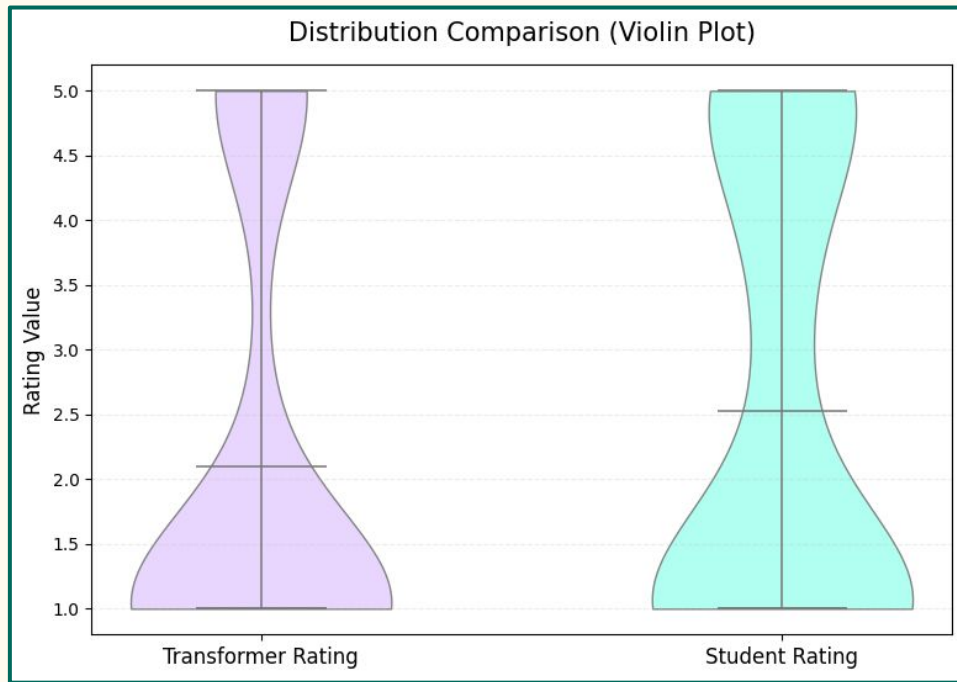
Using our newly trained weights. Prompt our model on each review and ask it for a rating. Then compare the rating from the transformer with our true rating from the student.



Model Performance

Best accuracy of 78.9%

With most accuracies around 70%. Here we can visualize the difference in distributions between our transformer and the real student ratings. From this we can see that our transformer tended to be more pessimistic (lower ratings) and more drawn towards the extremes of 5.0 and 1.0 most likely influenced by the prompt. Overall, this is a seemingly successful first attempt at building a model which can predict ratings from student reviews.



Where Does It Go Wrong?

The nature of planet terp leads to a different reviews of various intensity. Some reviews are mild mannered and contain pros and cons about a teacher while some let all their frustrations out in their review. I think this leads to some of the difficulty the model has as it's hard to differentiate a 1 star review with tons of curse words vs a 1 star review with genuine criticisms.

Another issue I found after looking through reviews that the model got wrong (by a margin greater than 1) is some reviews point their negativity at fellow students and the model treats this negativity as a poor review.

Model Rating: 1

Student Rating: 4

Never in my college experience have I come across a group of people who don't know that they don't know, take no responsibility, and then shit talk the instructor like they know better how

How can we improve?

- Training Improvement
- Prompt Improvement
- (Upgrade to Colab Pro)

More Training Details!

Unsurprisingly the more I trained the program the better the accuracy seemingly got. The drawback to this being training time started to get quite long and reach the limits of what Google Collab would allow. Future training with a greater epoch or batch size could lead to better accuracies making sure to watch out for overfitting.

Another aspect of the training would be to get more reviews. 406 reviews is a lot but just scratching the surface of what is on planet terp. More reviews, although would take longer, would allow our transformer to be even more fine tuned to the language used in these reviews and how they correspond to ratings.

Finally, a completely different model can be used to predict. Due to my own naivety I chose the first model I found that was easy to understand and had the parts needed to complete our task. Although it got the job done, further research into a more suitable model and training could hugely increase our success.

Prompt Engineering

Another aspect that could help with accuracies is prompt engineering the main prompt I used was:

'Give a single number of stars, between 1 and 5, that you think this student's review is assigning with a bias towards 1 or 5 stars '

This is a pretty basic when it comes to prompt and I'm sure accuracy could be increased by giving more details in the prompt. For instance, giving context behind where these reviews come from and who they are for as well as some added information behind how students tend to review their professors.

To Review

Successfully...

- Imported hundreds of student reviews using PlanetTerp API

- Used Microsoft's Phi-3 language model as our starting point.

- Trained our model using Supervised Fine-tune Training

- Prompted the model to provide a rating based on student written reviews

- Found moderate success in our model's ability to interpret reviews and predict ratings with an maximum accuracy of 78%

Work Cited

<https://huggingface.co/blog/dvgodoy/fine-tuning-llm-hugging-face>